Non-Primitive Data Type or Reference Data Types 1
Objects Oriented Programming 1
Class and Object 4
Example Blueprint 5
Instantiating: Making Objects from Your Class 5
What Happens in Computer Memory During Instantiation? 5
Encapsulation 6
Here is a complete, working example 7
Getter and Setter method 8
This - keyword 9
Inheritance 10
Abstract Class 12
Polymorphism 14
Polymorphism in Action (Code Example) 14
Variable scope 16

Non-Primitive Data Type or Reference Data Types

The Reference Data Types will contain a memory address of variable values because the reference types won't store the variable value directly in memory. They are strings, objects, arrays, etc.

Primitive data type vs. non-primitive type in Java

Primitive Datatypes	Non-Primitive Datatypes
Primitive Datatypes are pre-defined by the language itself.	Non-Primitive Datatypes need to be defined by the user.
When declared primitive data is stored in the stack.	The reference variables are stored in the stack but the original object is in the heap.
When a copy is created for the data, a completely separate allocation is done for the copy.	Copies here create new reference variables but they both point to the same object in the heap.
Thus, the changes made to the copy are not reflected in the original.	Here however the changes made to the copy trace back to the original.
The primitive data structure will contain some value, i.e., it cannot be NULL.	The non-primitive data structure can consist of a NULL value.
The size depends on the type of data structure.	In the case of a non-primitive data structure, size is not fixed.
The primitive data structure can be used to call the methods.	Non-primitive data structure cannot be used to call the methods.

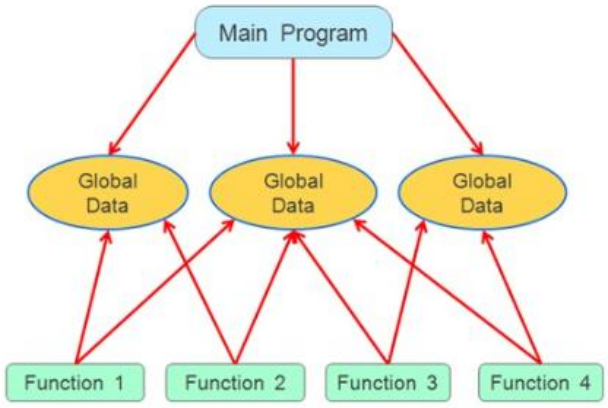
Objects Oriented Programming

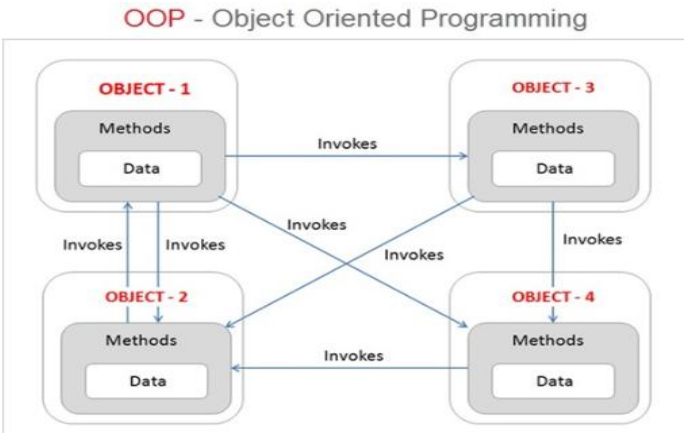
OOP, we can create modular, reusable, and maintainable code, making programming more efficient.

- In Procedural programming, it is difficult to secure data. Data is global and all functions have access to data.
- Fundamental concept of OOP is Class and Object. Classes serve as blueprints for creating objects.
- Each idea is modeled s class. Objects of classes interact to solve the problem.
- Other class does not have access to data of another ideas class. Data is protected and only member functions have access to data.

Video link: https://www.youtube.com/watch?v=m_MQYyJpIjg
https://www.youtube.com/watch?v=_x8Cyc9k4wk

Procedural programming

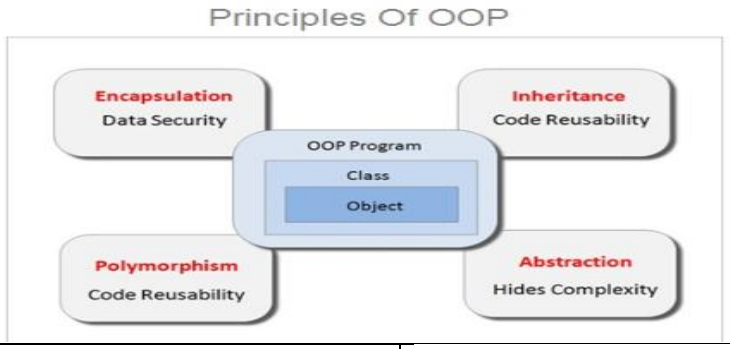




Principles Of Object Oriented Programming

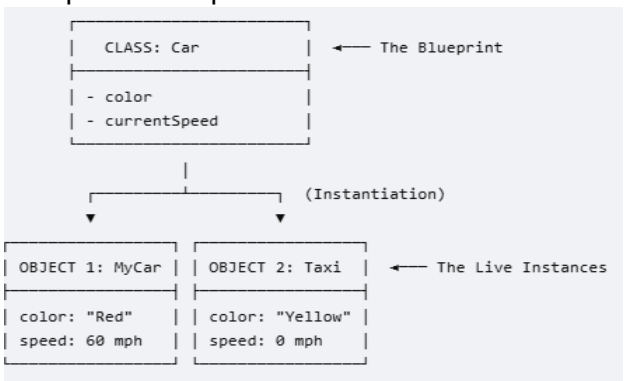
The OOP programming organization is based on the four major fundamental principles . The OOP principles include :

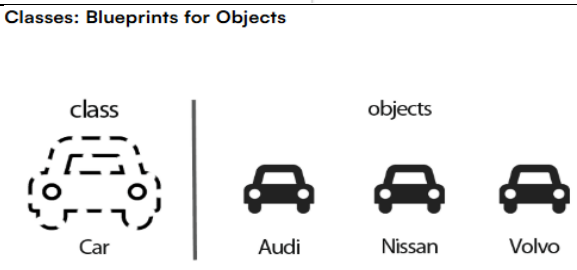
1. Encapsulation
2. Inheritance
3. Abstraction
4. Ploymorphism

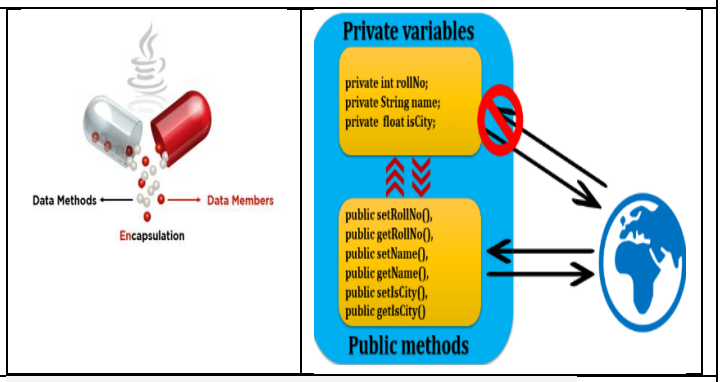
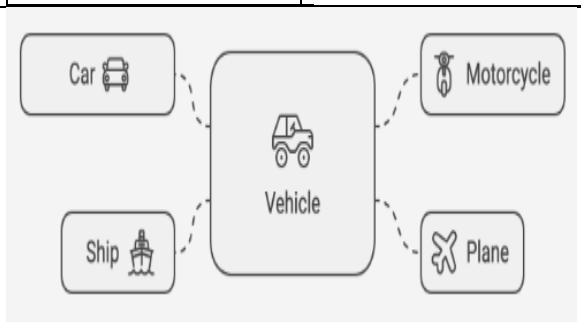
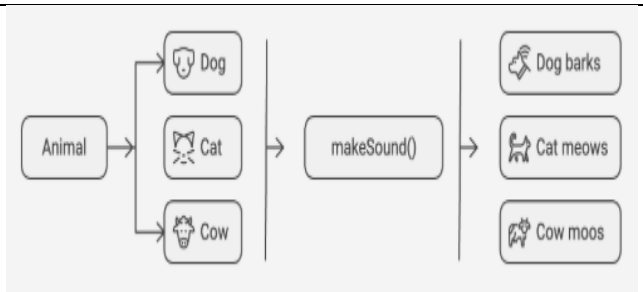


1> Abstraction

Abstraction: Simplifying Complexity by proving blueprint for expected behaviors and attributes.





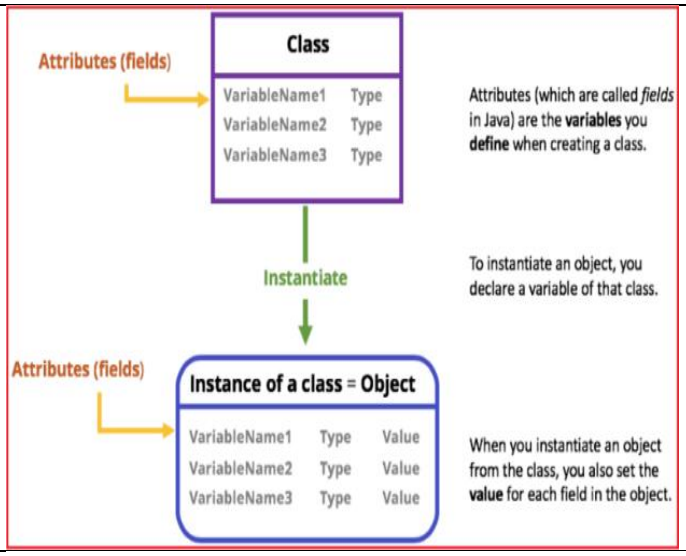
2. Encapsulation: Bundling Data and Logic *Powerful concept that combines data and logic into cohesive units. * Data hiding: Encapsulation safeguards the inner workings of objects, ensuring that they are accessible only through defined interfaces while hiding the intricate complexities. This is done by providing access modifier like Public, Private, Protected to data and methods	
3. Inheritance *Inheritance allows you to create new classes based on existing ones. * This helps **reduce code duplication** (reuse) and makes extending the code easier. *The new class (child) inherits all properties and methods from the **parent class** but can also add its own.	
4. Polymorphism **Polymorphism** allows objects of different classes to be treated as objects of a **common superclass**. It enables methods to do **different things** based on the object they are acting upon. It's like a **universal remote control** where one button can perform different functions depending on which device you're using.	

Class and Object

The Core Concept: Class and Object

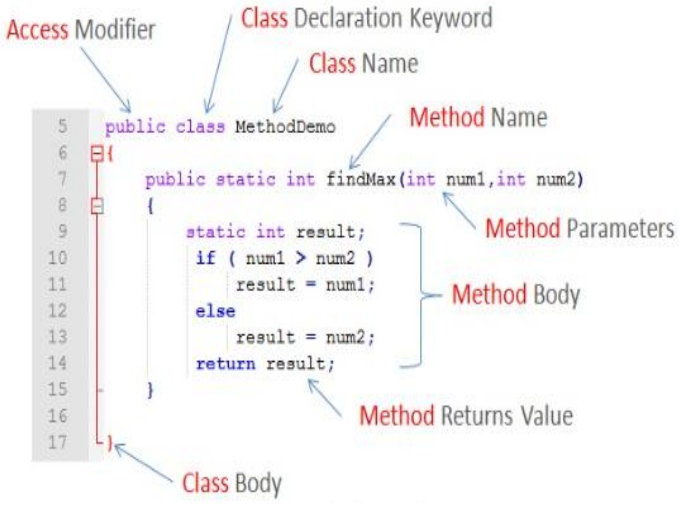
A Class is the Blueprint: It is the conceptual design, a template, or a factory mold. It does not physically exist; it just describes what something *will* look like and what it *will* do.

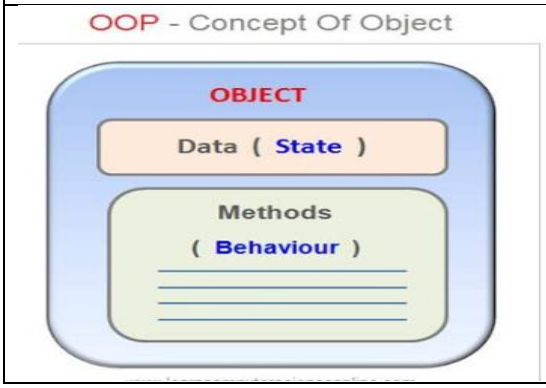
An Object is the Instance: It is the actual, physical item built directly from that blueprint. You can use one blueprint class to stamp out hundreds of individual objects

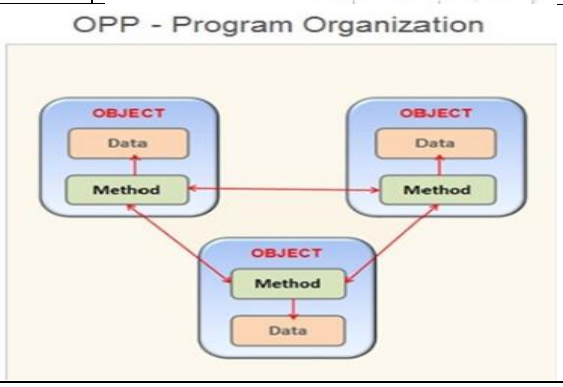


Anatomy of Class

- **Attributes (Fields / Instance Variables):** What the object *knows* or remembers about itself (its data). Properties define the characteristics or attributes of an object.
- **Methods:** What the object can *do* (its functional actions).
- **Constructor:** The initialization engine that assigns starting values to an object's fields upon birth.







Example Blueprint

```java
public class Car {
    // Attributes
    public String color;
    public int speed;

    // 1. THE CONSTRUCTOR (No return type, matches class name)
    public Car(String initialColor, int initialSpeed) {
        this.color = initialColor; // Sets the object's color to the incoming in
        this.speed = initialSpeed; // Sets the object's speed to the incoming in
    }

    // Method
    public void displayStatus() {
        System.out.println("The " + color + " car is going " + speed + " mph.");
    }
}
```

Instantiating: Making Objects from Your Class

new keyword in main( ) triggers the class constructor to build a live object in your computer's memory. instantiate and configure a object cleanly in a single line.

Dot Notation (.): The syntax syntax operator used to dig inside an object to access its variables or trigger its methods (e.g., p1.name).

```java
// This automatically jumps to the constructor code above!
Car myCar = new Car("Red", 60);
myCar.displayStatus(); // Prints: The Red car is going 60 mph.
```

What Happens in Computer Memory During Instantiation?

When Java executes the line Car genericCar = new Car("Red", 55);, it performs three distinct steps under the hood:

1. **new Car(...)**: Java allocates a brand-new chunk of blank memory big enough to hold a Car object.
2. **Car("Red", 55)**: The constructor function wakes up, takes your values, and writes "Red" into the color slot and 55 into the speed slot inside that fresh memory space.
3. **Car genericCar = ...**: Java assigns a variable pointer named genericCar to reference (point directly to) that specific spot in memory so you can find it later using dot notation.

Encapsulation

In Java, access modifiers control the visibility and accessibility of your classes, methods, variables, and constructors. They are the foundation of **encapsulation**, which allows you to secure your data and hide the inner workings of your code.
Here is a quick overview of how the three main modifiers, plus the "default" behavior, restrict access:

Access Modifier	Inside Same Class	Inside Same Package	Subclasses (Different Package)	The World (Everywhere)
`public`	✅ Yes	✅ Yes	✅ Yes	✅ Yes
`protected`	✅ Yes	✅ Yes	✅ Yes (via inheritance)	❌ No
default (no keyword)	✅ Yes	✅ Yes	❌ No	❌ No
`private`	✅ Yes	❌ No	❌ No	❌ No

Public (public)
The public modifier provides the **least restrictive** level of access. Anything declared as public can be seen and used by any other class in your entire project. [1, 2]

- **When to use:** Use this for your main API endpoints, major application services, helper utilities, or anything designed to be shared globally across packages.

Private (private)
The private modifier provides the **highest level of restriction**. Variables or methods marked private are hidden from every other class in your project—even classes inside the exact same folder (package). They can only be accessed within the specific class where they are defined. [1, 2, 3]

- **When to use:** Use this by default for almost all of your class variables (fields). You then expose them safely using public "getter" and "setter" methods if other classes need to read or change them.

Protected (protected)
The protected modifier is tailored specifically for inheritance. It allows a parent class to share its components with its children (subclasses), even if those child classes are located in completely different folders or packages. Additionally, any *other* class inside the same package can also see it.

- When to use: Use this when building a framework or class hierarchy where child classes genuinely need direct access to a parent's methods or data to extend its functionality.

Note on "Default" (Package-Private)

If you do not type public, private, or protected next to a variable or method, Java automatically assigns it **default** access. This means the code is visible to everything inside the **same package**, but entirely hidden from the outside world. [1, 2, 3]
Would you like to see a practical example of **how these modifiers interact** across different packages, or do you want to explore how **getters and setters** protect private fields? [1]

Here is a complete, working example

Here is the updated example split into **two distinct packages**: `vehicle` and `person`.
This setup demonstrates how the **`Main`** class interacts with both classes across package boundaries, highlighting how public members are accessible while private members remain hidden.

Project Directory Structure

To run this code locally, organize your files into folders matching the package names:

```
text

src/
│
├── vehicle/
│   └── Car.java
│
├── person/
│   └── Person.java
│
└── application/
    └── Main.java
```

```java
package vehicle;

public class Car {
    private String model;
    private int speed;

    // Public data member accessible from any package
    public String color;

    public String getModel() {
        return model;
    }

    public void setModel(String model) {
        this.model = model;
    }

    public int getSpeed() {
        return speed;
    }

    public void setSpeed(int speed) {
        if (speed >= 0) {
            this.speed = speed;
        }
    }
}
```

```java
package person;

public class Person {
    private String name;

    public Person(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // Interaction method: A Person drives a Car
    public void drive(vehicle.Car car) {
        System.out.println(name + " is driving a " + car.color + " " + car.getMo
    }
}
```

Getter and Setter method

A **getter method** is a public method used to safely retrieve (get) the value of a hidden, private variable from outside its class. By making class variables private, you prevent outside classes from accidentally changing or corrupting your data. The getter method acts as a controlled, **read-only window** to the world.

Why do this instead of just making the variable public?

1. **Read-Only Protection:** By providing only a getter (and no setter), you make the variable completely immutable to the outside world. Other classes can see it, but they can **never change it**.
2. **Data Logging & Control:** You can add logic inside the getter. For example, you could log a message every time a sensitive piece of data is accessed:

A **setter method** is a public method used to safely update or change (set) the value of a hidden, private variable from outside its class.

While getters provide read-only access, setters provide **write-only access**. By routing all changes through a setter method, your class can inspect, validate, or reject new values before updating the actual variable.

Why use a Setter instead of a public variable?

- **Data Validation:** You can block bad inputs (like preventing a setAge(int age) method from accepting negative numbers).
- **Encapsulation:** The internal storage logic is hidden. If you later decide to change how IDs are structured (e.g., adding a prefix automatically), you only change the code *inside* the setter, without breaking anyone else's code.

What Happens When You Run This?

```
package application;

// Importing classes from their respective packages
import vehicle.Car;
import person.Person;

public class Main {
   public static void main(String[] args) {
      // 1. Create a Car object
      Car myCar = new Car();
      myCar.setModel("Tesla Model 3");
      myCar.setSpeed(60);

      // Directly accessing the public data member from another package
      myCar.color = "Red";

      // 2. Create a Person object
      Person driver = new Person("Alice");

      // 3. Show interaction between Person and Car
      driver.drive(myCar);

      // Displaying speed using getter
      System.out.println(driver.getName() + "'s current speed: " + myCar.getSpeed() + " mph");
   }
}
```

This - keyword

How **this keyword** works inside getters and setters
In Java, the **this keyword** is a reference variable that refers directly to the **current object instance** you are working inside.
Inside getters and setters, its primary job is to **resolve naming conflicts** (called variable shadowing) and clarify exactly which variable you mean.

The Problem: Variable Shadowing

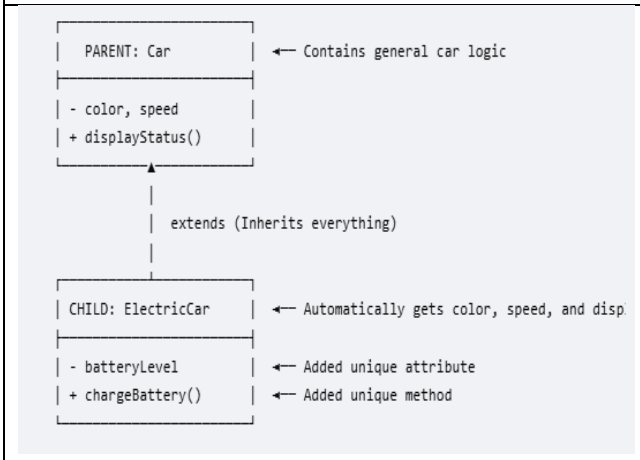
When you write a setter method, you usually name the incoming parameter the exact same name as the class variable.

public class Car {
private String model; // Instance variable
 	
public void setModel(String model) {
// Parameter has the same name
model = model;
 	
// ERROR/WARNING: You are assigning the parameter 'model' to itself!
// The instance variable above is ignored (shadowed).
}
}
	Without this, Java gets confused. It assumes both name references mean the local parameter (number 2) because it is closer in scope. This means the code effectively does nothing—it assigns the parameter to itself, leaving the actual class variable blank. This is called variable shadowing.

public class Car {
private String model; // Instance variable
 	
public void setModel(String model) {
this.model = model;
// SOLVED: 'this.model' belongs to the object.
 	
// 'model' belongs to the method parameter.
}
}
	The Solution: Using this
	To fix this, you prepend this. to the class variable. This explicitly tells Java: *"Hey, set the class-level variable belonging to this specific object equal to the local parameter value."*

Inheritance

Inheritance is one of the pillars of Object-Oriented Programming. It allows a new class to inherit (absorb) all the attributes and methods of an existing class automatically.
Think of inheritance like **genetic traits**. A child inherits traits from their parent, but the child can also have unique traits of their own.



- **Parent Class (Superclass):** The general, base class that holds common traits (e.g., Car).
- **Child Class (Subclass):** The specialized class that inherits everything from the parent but adds unique twists (e.g., an ElectricCar).
- **The Keyword: extends**

In Java, you use the **extends** keyword to establish an inheritance relationship.

- **@Override Annotation:** You place this helper tag right above the child method. It tells Java to double-check your code to ensure you didn't accidentally typo the name or parameters.

Why Use Inheritance?

- Code Reusability: You don't have to rewrite code for color, speed, or displayStatus() if you decide to build a GasCar, a Truck, or a RaceCar later. You just extend Car.
- While inheritance gives the child class all the parent's methods automatically, sometimes the default parent behavior isn't quite right for the child. **Method Overriding** occurs when a child class rewrites a method it inherited from its parent class to give it a new, specialized behavior. Child keep the exact same method name and inputs, but change what happens inside the execution brackets { }.
  - **Same Name:** The method name in the child class must exactly match the parent class.
  - **Same Parameters:** The inputs (arguments) must match exactly.

```
public class ElectricCar extends Car {
    public int batteryLevel;

    public ElectricCar(String color, int speed, int battery) {
        super(color, speed);
        this.batteryLevel = battery;
    }

    // ——— METHOD OVERRIDING HAPPENS HERE ———
    @Override
    public void displayStatus() {
        // We redefine the action specifically for electric cars
        System.out.println("The " + color + " electric car is going " + speed +
                           " mph with " + batteryLevel + "% battery remaining.");
    }
}
```

The `super` Shortcut in Overriding

Notice that inside the child's `displayStatus()`, we re-wrote the part printing the color and speed. If the parent method has a ton of complex code, re-writing it is a waste of time.

You can use `super.methodName()` to run the parent's version of the method *first*, and then add your child logic right below it:

java

```java
@Override
public void displayStatus() {
    super.displayStatus(); // Runs the parent version (Prints: The White car is 
    System.out.println("Battery Level: " + batteryLevel + "%"); // Adds unique ch
}
```

```java
public class Main {
    public static void main(String[] args) {

        Car basicCar = new Car("Red", 55);
        ElectricCar tesla = new ElectricCar("White", 70, 85);

        System.out.println("--- Testing Parent Method ---");
        basicCar.displayStatus();
        // Prints: The Red car is going 55 mph.

        System.out.println("\n--- Testing Overridden Child Method ---");
        tesla.displayStatus();
        // Prints: The White electric car is going 70 mph with 85% battery remai
    }
}
```

- Java is smart under the hood. Even though both objects are used. displayStatus(), Java looks at the specific object type at runtime and executes the customized version for the ElectricCar.
- If not Override, Java fall back to base class function.

Abstract Class

The Rules of Abstract Methods

1. **No Body:** Abstract methods end with a semicolon ; instead of curly braces {}.
2. **The abstract Keyword:** You must place the word abstract before the method name.
3. **Abstract Class Requirement:** If a class contains even *one* abstract method, **the entire class must be declared as abstract**. You cannot instantiate (create a new) abstract class directly.
4. **Child Obligation:** Any normal child class that inherits from an abstract parent **must override and write the body** for all abstract methods, or your code will fail to compile.

The Abstract Parent Class

Traditional gas car accelerates by burning fuel, while an electric car accelerates by pulling power from a battery pack. Instead of writing a generic, useless default method in the parent class, we make it **abstract**.

```
// We add 'abstract' to the class because it contains an abstract method
public abstract class Vehicle {
    public String color;

    public Vehicle(String color) {
        this.color = color;
    }

    // Standard method: Inherited normally by all children
    public void honk() {
        System.out.println("Beep beep!");
    }

    // ABSTRACT METHOD: No curly braces! Just a rule for the children.
    public abstract void accelerate();
}
```

The Child Classes (Fulfilling the Contract)

Now, any child class that extends Vehicle is forced by Java to implement accelerate(). If you forget to add it, Java will give you a compilation error.

```
public class GasCar extends Vehicle {
    public GasCar(String color) {
        super(color);
    }

    // We MUST override and provide a body for the abstract method
    @Override
    public void accelerate() {
        System.out.println("The gas engine revs! Fuel is injected into cylinders
    }
}

public class ElectricCar extends Vehicle {
    public ElectricCar(String color) {
        super(color);
    }

    // We MUST override and provide a body for the abstract method too
    @Override
    public void accelerate() {
        System.out.println("The electric motor hums instantly! Battery power sur
    }
}
```

Running the Code in **Main**

Remember, you **cannot** do Vehicle v = new Vehicle("Red"); because abstract classes are incomplete blueprints. You can only instantiate the concrete children.

```
public class Main {
    public static void main(String[] args) {

        // Instantiate the concrete child classes
        GasCar mustang = new GasCar("Blue");
        ElectricCar tesla = new ElectricCar("White");

        // Both can use the normal inherited method
        mustang.honk(); // Prints: Beep beep!

        // Each executes its own mandatory version of the abstract method
        mustang.accelerate(); // Prints: The gas engine revs!...
        tesla.accelerate();   // Prints: The electric motor hums!...
    }
}
```

Why Use Abstract Methods?

Abstract methods allow you to enforce a standard design structure across multiple different classes. It guarantees that no matter what kind of vehicle someone programs in the future, it is guaranteed to have an accelerate() action built into it.

Polymorphism

Polymorphism means "many forms." In Java, it allows you to treat different child objects (`GasCar` , `ElectricCar`) as if they were their parent type (`Vehicle`) [sports]. This is powerful because it lets you write one piece of code that works automatically with any type of vehicle you create in the future.

The Big Magic: Upcasting

Polymorphism allows you to declare a variable using the **Parent type**, but instantiate it using a **Child type**.

```java
// Declared as a Vehicle, but instantiated as a GasCar
Vehicle myRide = new GasCar("Blue");
```

Even though the variable type on the left is Vehicle, Java looks at the real object on the right (GasCar) at runtime and executes the GasCar version of your methods.

Polymorphism in Action (Code Example)

Let's look at how polymorphism allows us to manage all our cars inside a single array loop and pass them into a single function.

Why is this so useful?
Imagine you are building a video game with 100 different types of cars. Without polymorphism, you would have to write 100 separate functions to start a race (startGasCarRace(), startElectricCarRace(), etc.). With polymorphism, you write **one single function** that accepts Vehicle, and it handles all 100 car types perfectly.

What Prints to the Console?

```
--- Testing Loop Polymorphism ---
The gas engine revs! Fuel is injected into cylinders.
The electric motor hums instantly! Battery power surges.

--- Testing Function Polymorphism ---
The Red vehicle prepares... The gas engine revs! Fuel is injected into cylinders.
The Black vehicle prepares... The electric motor hums instantly! Battery power surges.
```

Polymorphism in Action (Code Example)

```
public class Main {
    public static void main(String[] args) {

        // 1. POLYMORPHISM IN ARRAYS
        // We create an array of type Vehicle. It can hold ANY child of Vehicle!
        Vehicle[] fleet = new Vehicle[2];

        fleet[0] = new GasCar("Blue");     // Upcasted automatically
        fleet[1] = new ElectricCar("White"); // Upcasted automatically

        System.out.println("--- Testing Loop Polymorphism ---");
        for (Vehicle v : fleet) {
            // Java automatically figures out which version of accelerate() to run!
            v.accelerate();
        }

        System.out.println("\n--- Testing Function Polymorphism ---");
        GasCar mustang = new GasCar("Red");
        ElectricCar tesla = new ElectricCar("Black");

        // 2. POLYMORPHISM IN FUNCTIONS
        // We can pass different object types into the exact same function
        startRace(mustang);
        startRace(tesla);
    }

    // This function accepts a generic Vehicle parent type
    public static void startRace(Vehicle v) {
        System.out.print("The " + v.color + " vehicle prepares... ");
        v.accelerate(); // Polymorphic method call
    }
}
```

Variable scope

Different types of variables are

- Local Variables
- Instance Variables
- Static Variables

Local Variables
A variable defined within a block or method or constructor is called a local variable.
• These variables are created when the block is entered, or the function is called and destroyed after exiting from the block or when the call returns from the function.
• The scope of these variables exists only within the block in which the variables are declared, i.e., we can access these variables only within that block.
• Initialization of the local variable is mandatory before using it in the defined scope.
	Class hcl guvi {
	public static void main(String[] args)
	{
	// Declared a Local Variable
	int var = 10;
	// This variable is local to this main method only
	System.out.println("Local Variable: " + **var**);
	}
	}

Instance variables are non-static variables and are declared in a class outside of any method, constructor, or block.
• As instance variables are declared in a class, these variables are created when an object of the class is created and destroyed when the object is destroyed.
• Unlike local variables, we may use access specifiers for instance variables. If we do not specify any access specifier, then the default access specifier will be used.
• Initialization of an instance variable is not mandatory. Its default value is dependent on the data type of variable.
• Instance variables can be accessed only by creating objects.
• We initialize instance variables using constructors while creating an object. We can also use instance blocks to initialize the instance variables.
	// Java Program to demonstrateclass
	hcl guvi {
	// Declared instance variable
	String student = "Shubham Jain";
	public static void main(String[] args) {
	HCL Guvi Zen=new HCLGuvi();
	// geek variable can be accessed by creating object
	System.out.print(Zen. student);
	}
	}